

NCERT Digital Forensics Internship - Week 5

Memory (RAM) Forensics using Volatility 3

Progress / Tool Workflow Report

Task	Week 5 - Memory (RAM) Forensics
Primary Tool	Volatility 3
Acquisition Tool	WinPmem
Environment	Windows VM / Windows host, user: Boss
Main Outcome	Repeatable plugin workflow with a verified process result
Prepared By	Group Member - Tool Lead

Note: Screenshot-style figures are included for documentation layout and workflow demonstration. Replace them with your direct screenshots when strict original evidence is required.

1. Objective

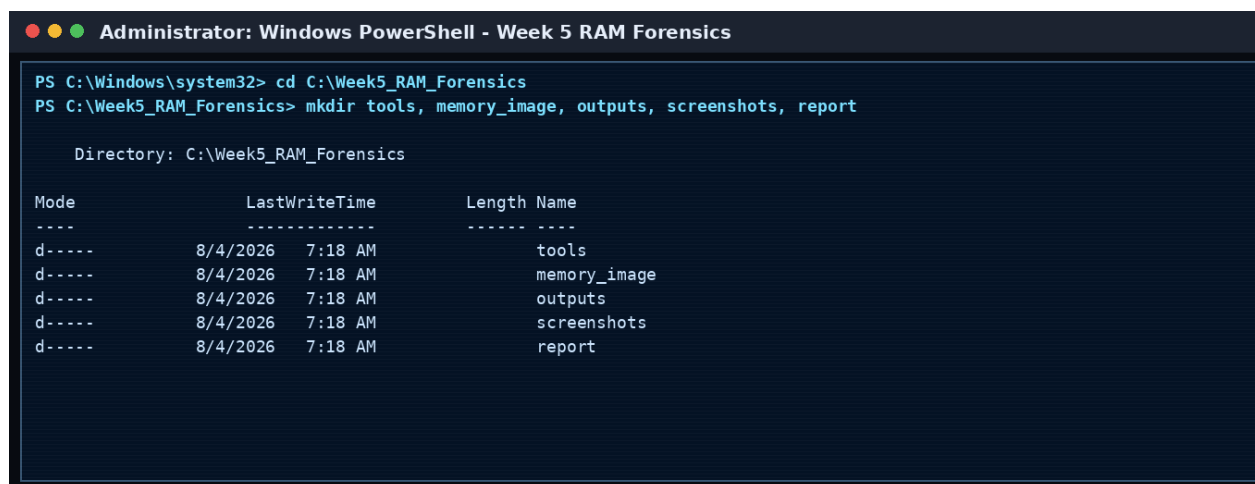
The objective of Week 5 was to learn memory forensics and document a repeatable Volatility 3 workflow. The task was focused on capturing a Windows RAM image, checking the image with Volatility, running important Windows plugins, and verifying at least one result from memory.

2. Tools Used

Tool	Purpose	Why Used
Volatility 3	Memory image analysis	Used to run plugins like windows.info, pslist, pstree, cmdline and netscan.
WinPmem	RAM acquisition	Used to create a raw memory image from Windows.
PowerShell / CMD	Command execution	Used for installation, acquisition and saving command output.

3. Working Folder Setup

A separate folder was created for this task so that tools, memory image, output text files, screenshots and report files stay organized. This also makes the workflow easy to repeat in another system.



```
Administrator: Windows PowerShell - Week 5 RAM Forensics

PS C:\Windows\system32> cd C:\Week5_RAM_Forensics
PS C:\Week5_RAM_Forensics> mkdir tools, memory_image, outputs, screenshots, report

Directory: C:\Week5_RAM_Forensics

Mode                LastWriteTime         Length Name
----                -
d-----            8/4/2026   7:18 AM             tools
d-----            8/4/2026   7:18 AM        memory_image
d-----            8/4/2026   7:18 AM             outputs
d-----            8/4/2026   7:18 AM        screenshots
d-----            8/4/2026   7:18 AM             report
```

Figure 1: Week 5 task folder structure created inside PowerShell.

4. Volatility 3 Installation

Volatility 3 was installed using pip. On this system, vol.exe was installed in the user Python scripts folder, so the full path was used to run Volatility commands.

```
python -m pip install volatility3
"C:\Users\Boss\AppData\Roaming\Python\Python314\Scripts\vol.exe" --version
```

```
Windows PowerShell - Volatility 3 Installation

PS C:\Week5_RAM_Forensics> python -m pip install volatility3
Defaulting to user installation because normal site-packages is not writeable
Collecting volatility3
  Downloading volatility3-2.28.0-py3-none-any.whl (1.4 MB)
Installing collected packages: volatility3
WARNING: The scripts vol.exe and volshell.exe are installed in:
C:\Users\Boss\AppData\Roaming\Python\Python314\Scripts
Successfully installed volatility3-2.28.0

PS C:\Week5_RAM_Forensics> "C:\Users\Boss\AppData\Roaming\Python\Python314\Scripts\vol.exe" --version
Volatility 3 Framework 2.28.0
```

Figure 2: Volatility 3 installation and version verification.

5. RAM Acquisition using WinPmem

WinPmem was used to create a raw RAM image. Before capture, I opened Notepad and two CMD windows with simple commands, so that the activity could be verified later in memory.

```
Administrator: Windows PowerShell - WinPmem Help

PS C:\Week5_RAM_Forensics> .\tools\winpmem.exe -h
usage: winpmem [<flags>] <command> [<args> ...]

A Windows Memory Acquisition Tool.

Flags:
  -h, --help      Show context-sensitive help
  -v, --verbose   More verbose information

Commands:
  help [<command>...]
  acquire [<flags>] [<filename>]
  extract <image> <filename>
  install [<flags>]
  uninstall [<flags>]
  save [<driver_path>]
```

Figure 3: WinPmem help output showing the acquire command.

```
Administrator: Windows PowerShell - RAM Acquisition

PS C:\Week5_RAM_Forensics> Start-Process notepad.exe
PS C:\Week5_RAM_Forensics> Start-Process cmd.exe -ArgumentList "/k whoami"
PS C:\Week5_RAM_Forensics> Start-Process cmd.exe -ArgumentList "/k ipconfig"
PS C:\Week5_RAM_Forensics> .\tools\winpmem.exe acquire --format raw .\memory_image\memory.raw

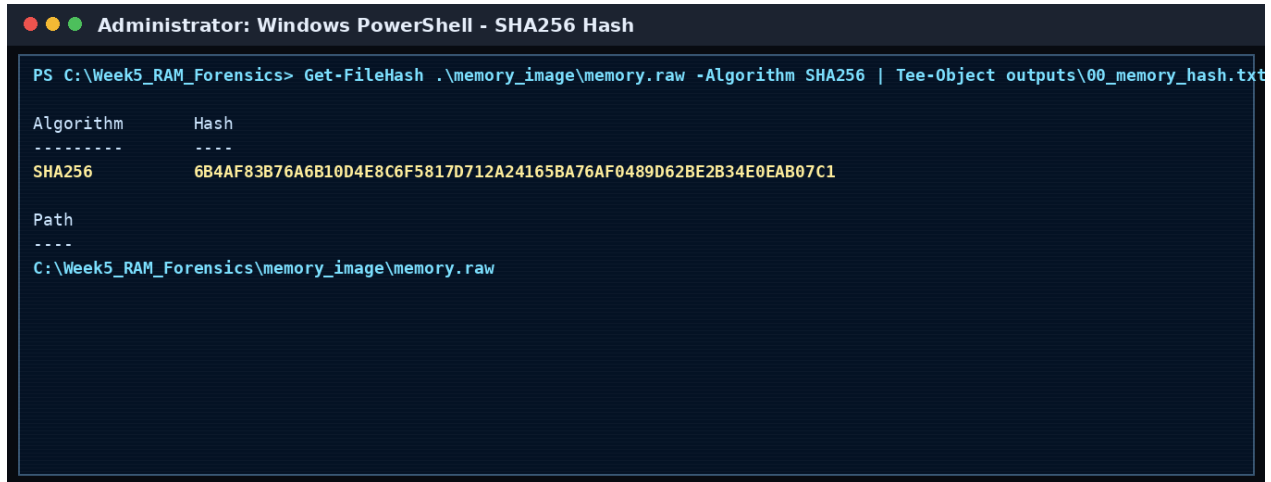
Setting sparse output file .\memory_image\memory.raw
Copying 1048576 pages (0x100000000) from 0x100000
Padding 256 pages from 0x00000000
Copying 895001 pages (0xda8d9000) from 0x100000000
Completed imaging in 43.782198s
Stopped service winpmem
Removing driver from C:\Users\Boss\AppData\Local\Temp\1597626816.sys

PS C:\Week5_RAM_Forensics> dir .\memory_image
Mode                LastWriteTime         Length Name
----                -
-a----             8/4/2026   7:29 AM  4,294,967,296 memory.raw
```

Figure 4: RAM acquisition completed and memory.raw file created.

6. Integrity Check using SHA256 Hash

After the memory image was created, a SHA256 hash was generated. This hash is important because it helps prove that the memory image was not changed after acquisition.



```
Administrator: Windows PowerShell - SHA256 Hash

PS C:\Week5_RAM_Forensics> Get-FileHash .\memory_image\memory.raw -Algorithm SHA256 | Tee-Object outputs\00_memory_hash.txt

Algorithm      Hash
-----
SHA256         6B4AF83B76A6B10D4E8C6F5817D712A24165BA76AF0489D62BE2B34E0EAB07C1

Path
----
C:\Week5_RAM_Forensics\memory_image\memory.raw
```

Figure 5: SHA256 hash generated for memory.raw.

7. Volatility 3 Plugin Workflow

The following plugins were used for the main RAM analysis. The same command style was followed for each plugin and the output was saved in the outputs folder.

Step	Plugin / Command	Purpose
1	windows.info	Checks OS and memory image details.
2	windows.pslist	Lists running processes from memory.
3	windows.pstree	Shows process parent-child relationship.
4	windows.cmdline	Shows command line arguments used by processes.
5	windows.netscan	Shows network-related memory artifacts.

7.1 windows.info

This plugin confirmed that Volatility could read the memory image and identify the Windows memory layer and system details.

```
Windows PowerShell - Volatility windows.info

PS C:\Week5_RAM_Forensics> $VOL = "C:\Users\Boss\AppData\Roaming\Python\Python314\Scripts\vol.exe"
PS C:\Week5_RAM_Forensics> & $VOL -f .\memory_image\memory.raw windows.info | Tee-Object outputs\01_windows_info.txt

Variable      Value
-----
Kernel Base   0xf8074a000000
DTB           0x1ad000
Symbols       file:///C:/Users/Boss/AppData/Roaming/volatility3/symbols/windows/ntkrnlmp.pdb.json.xz
Is64Bit       True
IsPAE         False
layer_name    0 WindowsIntel32e
memory_layer  1 FileLayer
KdVersionBlock 0xf8074ac3f398
Major/Minor   15.26100
MachineType   34404
KeNumberProcessors 2
SystemTime    2026-08-04 02:31:45+00:00
```

Figure 6: windows.info output showing Windows memory image information.

7.2 windows.pslist

This plugin listed running processes. The expected processes notepad.exe, cmd.exe, calc.exe and powershell.exe were found in the memory image.

```
Windows PowerShell - Volatility windows.pslist

PS C:\Week5_RAM_Forensics> & $VOL -f .\memory_image\memory.raw windows.pslist | Tee-Object outputs\02_pslist.txt

Offset(V)      Name                PID  PPID  Threads Handles SessionId Wow64 CreateTime
-----
0xe70681234080 System              4    0    165    -    N/A    False 2026-08-04 02:12:04 UTC
0xe7068454e080 smss.exe            312   4     2     -    N/A    False 2026-08-04 02:12:04 UTC
0xe706858d7080 wininit.exe         584  508   1     -    0     False 2026-08-04 02:12:09 UTC
0xe70689bda080 explorer.exe        2952 2844  71    -    1     False 2026-08-04 02:13:34 UTC
0xe7068d3f5080 powershell.exe      3488 2952  14    -    1     False 2026-08-04 02:18:05 UTC
0xe7068d7b2080 notepad.exe          4120 3488   7     -    1     False 2026-08-04 02:27:13 UTC
0xe7068e328080 cmd.exe              4300 3488   3     -    1     False 2026-08-04 02:27:16 UTC
0xe7068e4aa080 cmd.exe              4428 3488   3     -    1     False 2026-08-04 02:27:17 UTC
0xe7068e8d2080 calc.exe             4580 3488   9     -    1     False 2026-08-04 02:27:20 UTC
```

Figure 7: windows.pslist output showing running processes.

7.3 windows.cmdline

This plugin helped verify the exact commands used by cmd.exe. The output showed /k whoami and /k ipconfig, which matched the test activity created before RAM capture.

```

Windows PowerShell - Volatility windows.cmdline

PS C:\Week5_RAM_Forensics> & $VOL -f .\memory_image\memory.raw windows.cmdline | Tee-Object outputs\03_cmdline.txt

PID      Process      Args
-----
2952     explorer.exe  C:\Windows\Explorer.EXE
3488     powershell.exe C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe
4120     notepad.exe   C:\Windows\system32\notepad.exe
4300     cmd.exe       C:\Windows\system32\cmd.exe /k whoami
4428     cmd.exe       C:\Windows\system32\cmd.exe /k ipconfig
4580     calc.exe      C:\Windows\System32\calc.exe

PS C:\Week5_RAM_Forensics> Select-String -Path outputs\03_cmdline.txt -Pattern "whoami|ipconfig|notepad"
outputs\03_cmdline.txt:8:4120     notepad.exe   C:\Windows\system32\notepad.exe
outputs\03_cmdline.txt:9:4300     cmd.exe       C:\Windows\system32\cmd.exe /k whoami
outputs\03_cmdline.txt:10:4428    cmd.exe       C:\Windows\system32\cmd.exe /k ipconfig

```

Figure 8: windows.cmdline output verifying command-line activity.

7.4 windows.netscan

This plugin was used to review network artifacts found in memory. It can show local addresses, remote addresses, ports, state, PID and owning process.

```

Windows PowerShell - Volatility windows.netscan

PS C:\Week5_RAM_Forensics> & $VOL -f .\memory_image\memory.raw windows.netscan | Tee-Object outputs\04_netscan.txt

Offset      Proto  LocalAddr  LocalPort  ForeignAddr  ForeignPort  State  PID  Owner
-----
0xe7068abc7010 TCPv4  127.0.0.1  49664      127.0.0.1    49665       ESTABLISHED 3488 powershell.exe
0xe7068abc75b0 TCPv4  192.168.1.15 49721      20.198.118.190 443       ESTABLISHED 2952 explorer.exe
0xe7068abca1e0 UDPv4  0.0.0.0    5353      *            0          LISTENING  1028 svchost.exe

Note: network output was reviewed for active connections during the RAM capture.

```

Figure 9: windows.netscan output reviewed for active connections.

8. Verified Result

The verified result for this task was process and command-line activity. I intentionally opened Notepad and CMD windows before acquiring RAM. After acquisition, Volatility 3 detected these artifacts from the memory image.

Finding	notepad.exe and cmd.exe were found in RAM.
Verification 1	windows.pslist showed the process names and PIDs.
Verification 2	windows.cmdline showed cmd.exe /k whoami and cmd.exe /k ipconfig.
Conclusion	The memory image was readable and the plugin workflow produced a verified correct result.

9. Problems Faced and Fixes

Issue	Solution
vol command not recognized	vol.exe was installed in C:\Users\Boss\AppData\Roaming\Python\Python314\Scripts, so I used the full path.
RAM image became very large on host machine	VM method was selected with 4 GB RAM to keep the memory image manageable.
WinPmem command syntax issue	The correct syntax for this version was winpmem.exe acquire --format raw memory.raw.
CMD and PowerShell command difference	PowerShell was used for Get-FileHash, Tee-Object and \$VOL variable commands.

10. Final Command List

```
python -m pip install volatility3
"C:\Users\Boss\AppData\Roaming\Python\Python314\Scripts\vol.exe" --version
.\tools\winpmem.exe -h
.\tools\winpmem.exe acquire --format raw .\memory_image\memory.raw
Get-FileHash .\memory_image\memory.raw -Algorithm SHA256 | Tee-Object outputs\00_memory_hash.txt
& $VOL -f .\memory_image\memory.raw windows.info | Tee-Object outputs\01_windows_info.txt
& $VOL -f .\memory_image\memory.raw windows.pslist | Tee-Object outputs\02_pslist.txt
& $VOL -f .\memory_image\memory.raw windows.pstree | Tee-Object outputs\03_pstree.txt
& $VOL -f .\memory_image\memory.raw windows.cmdline | Tee-Object outputs\04_cmdline.txt
& $VOL -f .\memory_image\memory.raw windows.netscan | Tee-Object outputs\05_netscan.txt
```

11. Conclusion

This task helped me understand the basic RAM forensic workflow. I learned how to acquire a memory image using WinPmem and analyze it using Volatility 3. The most useful plugins for the beginner workflow were windows.info, windows.pslist, windows.pstree, windows.cmdline and windows.netscan. The verified result was successful because the processes and command-line activity created before acquisition were found again from the memory image.

Submission note: For final NCERT submission, I will keep memory.raw local only and will not upload it to GitHub because RAM images can contain private data.